

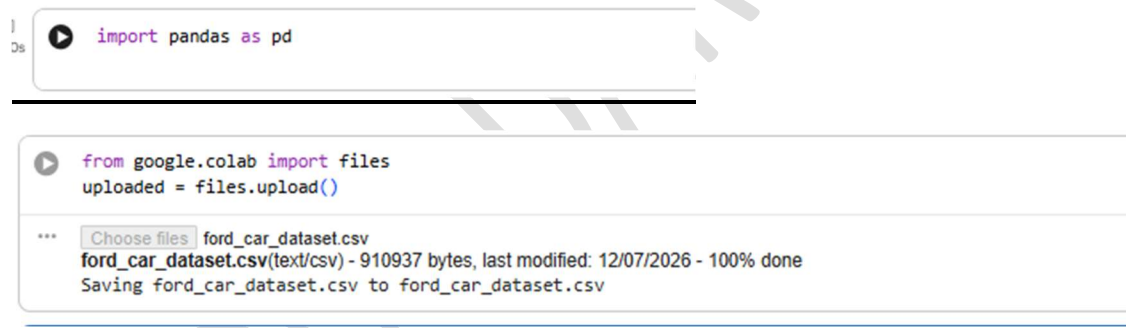
## Session 18 (AIML) - Assignment Question

- **Name of Student :** Shruti Dnyaneshwar Darwatkar
- **Domain Name:** AIML Development
- **College Name:** Zeal Polytechnic , Narhe
- **Branch Name:** AIML
- **Github url:** [https://github.com/shrutidarwatkar-sketch/Task\\_ITR.git](https://github.com/shrutidarwatkar-sketch/Task_ITR.git)

**Q1. Data Loading & Initial Analysis) Load the Ford Car Dataset using pandas.**

- Display the first 10 rows and last 5 rows.
- Check the shape (number of rows and columns) and data types of all columns.
- Write observations about the dataset structure in comments

### PROGRAM SCREENSHOT :



```
1 | import pandas as pd
```

```
from google.colab import files
uploaded = files.upload()

*** Choose files: ford_car_dataset.csv
ford_car_dataset.csv(text/csv) - 910937 bytes, last modified: 12/07/2026 - 100% done
Saving ford_car_dataset.csv to ford_car_dataset.csv
```

```

# Load dataset
df = pd.read_csv('/content/ford_car_dataset.csv')

# First 10 rows
print(df.head(10))

# Last 5 rows
print(df.tail())

# Shape
print("Shape:", df.shape)

# Data types
print(df.dtypes)

# Observation:
# The dataset contains both numerical and categorical columns.

```

|   | model  | year | price | transmission | mileage | fuelType | tax | mpg  | engineSize |
|---|--------|------|-------|--------------|---------|----------|-----|------|------------|
| 0 | Fiesta | 2017 | 12000 | Automatic    | 15944   | Petrol   | 150 | 57.7 | 1.0        |
| 1 | Focus  | 2018 | 14000 | Manual       | 9083    | Petrol   | 150 | 57.7 | 1.0        |
| 2 | Focus  | 2017 | 13000 | Manual       | 12456   | Petrol   | 150 | 57.7 | 1.0        |
| 3 | Fiesta | 2019 | 17500 | Manual       | 10460   | Petrol   | 145 | 40.3 | 1.5        |
| 4 | Fiesta | 2019 | 16500 | Automatic    | 1482    | Petrol   | 145 | 48.7 | 1.0        |
| 5 | Fiesta | 2015 | 10500 | Manual       | 35432   | Petrol   | 145 | 47.9 | 1.6        |
| 6 | Puma   | 2019 | 22500 | Manual       | 2029    | Petrol   | 145 | 50.4 | 1.0        |
| 7 | Fiesta | 2017 | 9000  | Manual       | 13054   | Petrol   | 145 | 54.3 | 1.2        |
| 8 | Kuga   | 2019 | 25500 | Automatic    | 6894    | Diesel   | 145 | 42.2 | 2.0        |
| 9 | Focus  | 2018 | 10000 | Manual       | 48141   | Petrol   | 145 | 61.4 | 1.0        |

```

17961 B-MAX 2017 8999 Manual 16700 Petrol 150 47.1
17962 B-MAX 2014 7499 Manual 40700 Petrol 30 57.7
17963 Focus 2015 9999 Manual 7010 Diesel 20 67.3
17964 KA 2018 8299 Manual 5007 Petrol 145 57.7
17965 Focus 2015 8299 Manual 5007 Petrol 22 57.7

engineSize
17961 1.4
17962 1.0
17963 1.6
17964 1.2
17965 1.0
Shape: (17966, 9)
model      object
year       int64
price      int64
transmission object
mileage    int64
fuelType   object
tax        int64
mpg        float64
engineSize float64
dtypes: object

```

## Q2. Missing & Duplicate Values) Perform a detailed check on the dataset:

- Find the total number of missing values in each column.
- Identify and remove duplicate rows if any.
- Write a summary of your findings and how you handled them in comments.

## PROGRAM SCREENSHOT :

```

#Q2
# Missing values
print(df.isnull().sum())

# Duplicate rows
print("Duplicate Rows:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()

print("New Shape:", df.shape)

# Observation:
# Duplicate rows are removed.

```

|   | model | year | price | transmission | mileage | fuelType | tax | mpg | engineSize |
|---|-------|------|-------|--------------|---------|----------|-----|-----|------------|
| 0 |       |      |       |              |         |          |     |     |            |
| 1 |       |      |       |              |         |          |     |     |            |
| 2 |       |      |       |              |         |          |     |     |            |
| 3 |       |      |       |              |         |          |     |     |            |
| 4 |       |      |       |              |         |          |     |     |            |
| 5 |       |      |       |              |         |          |     |     |            |
| 6 |       |      |       |              |         |          |     |     |            |
| 7 |       |      |       |              |         |          |     |     |            |
| 8 |       |      |       |              |         |          |     |     |            |
| 9 |       |      |       |              |         |          |     |     |            |

```

dtypes: int64
Duplicate Rows: 154
New Shape: (17812, 9)

```

### Q3. (Statistical Summary)

Generate the statistical summary of all numeric columns using describe().

- Identify minimum, maximum, mean, and median values for key columns like price, mileage, and year.

#### PROGRAM SCREENSHOT :

```
#Q3

# Statistical summary
print(df.describe())

# Key columns
columns = ['price', 'mileage', 'year']

for col in columns:
    print("\nColumn:", col)
    print("Minimum:", df[col].min())
    print("Maximum:", df[col].max())
    print("Mean:", df[col].mean())
    print("Median:", df[col].median())
```

---

```
...
count    17812.000000    17812.000000    17812.000000    17812.000000    17812.000000
mean     2016.862396    12269.556310    23381.146362    113.315012     57.908696
std        2.052039     4736.285417    19419.011045     62.034603    10.132696
min       1996.000000     495.000000     1.000000     0.000000    20.800000
25%       2016.000000     8999.000000    10000.000000     30.000000    52.300000
50%       2017.000000    11288.000000    18277.000000    145.000000    58.900000
75%       2018.000000    15295.000000    31098.500000    145.000000    65.700000
max       2060.000000    54995.000000   177644.000000    580.000000   201.800000

engineSize
count    17812.000000
mean         1.350623
std         0.432581
min          0.000000
25%          1.000000
50%          1.200000
75%          1.500000
max          5.000000

Column: price
Minimum: 495
Maximum: 54995
Mean: 12269.556310352571
Median: 11288.0

Column: mileage
Minimum: 1
Maximum: 177644
Mean: 23381.146362003143
Median: 18277.0

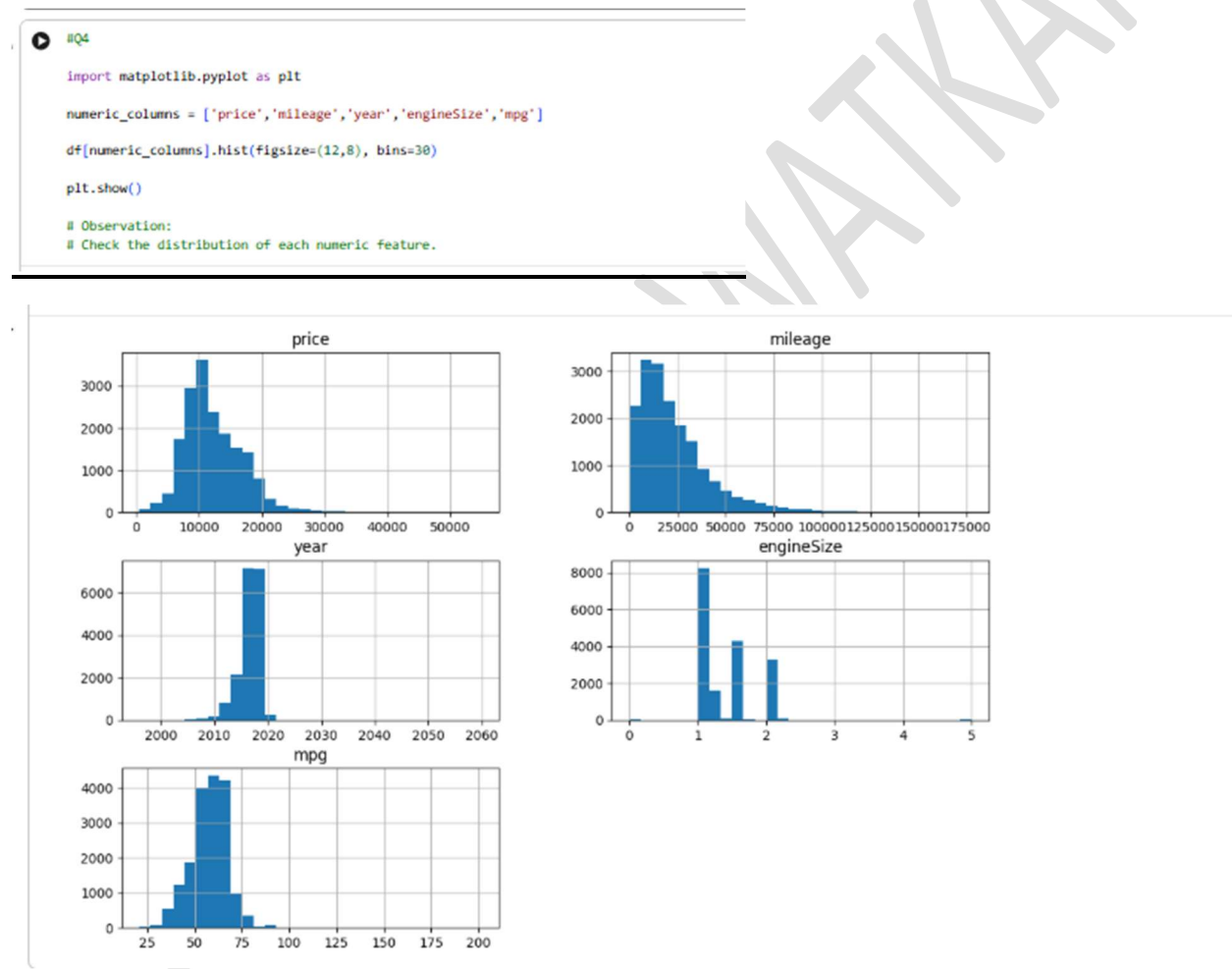
Column: year
Minimum: 1996
Maximum: 2060
Mean: 2016.8623961374353
Median: 2017.0
```

#### Q4. Histogram of Numeric Features)

Plot histograms for all important numeric columns (price, mileage, year, engine Size, mpg).

- Analyze the distribution for each feature and note your findings in comments.

#### PROGRAM SCREENSHOT :



## Q5. Count Plots of Categorical Features)

Create count plots for all categorical columns (fuel Type, transmission, model, etc.) using seaborn.

- Analyze which categories are most common.
- Write insights about market trends based on these plots in comments.

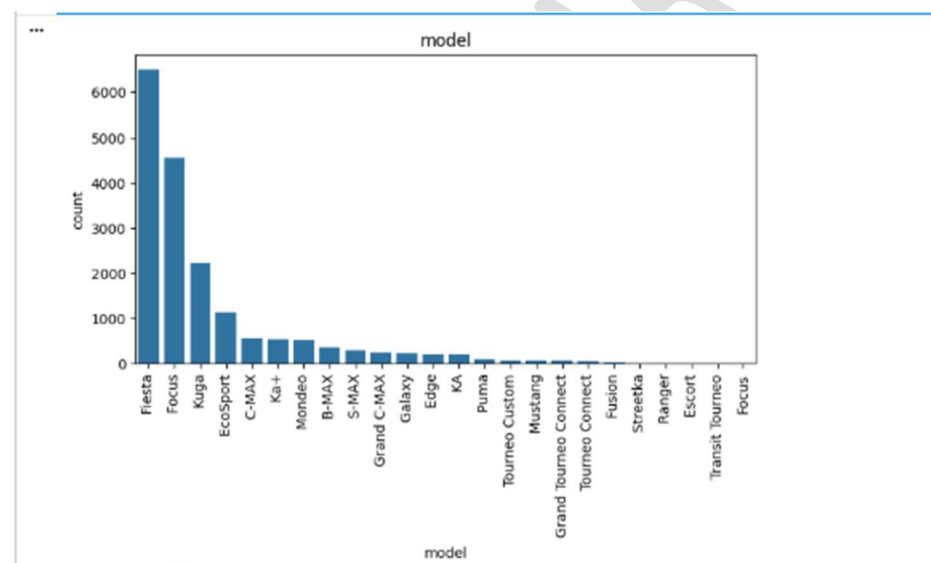
### PROGRAM SCREENSHOT :

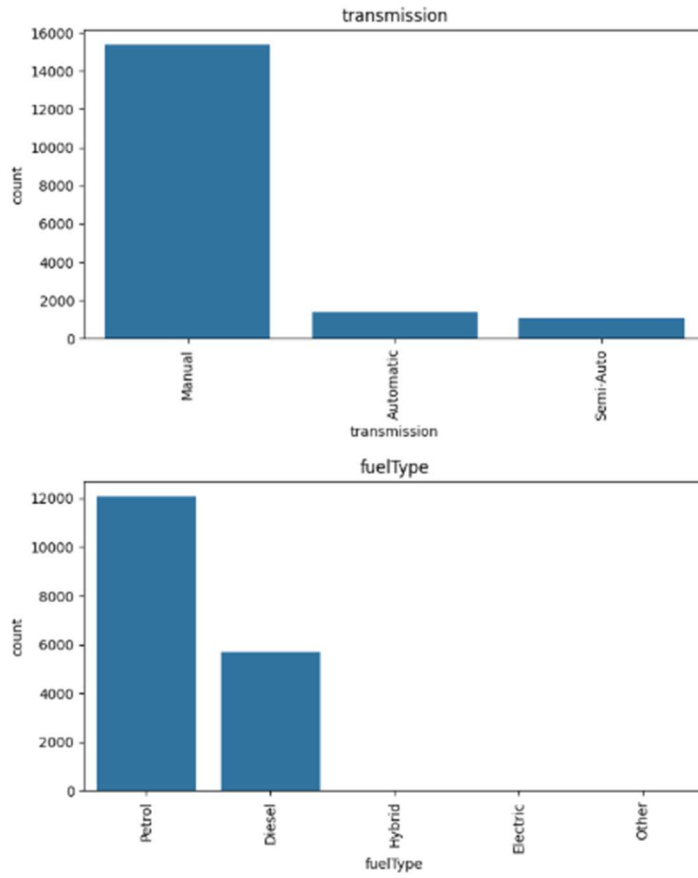
```
#Q5
import seaborn as sns
import matplotlib.pyplot as plt

categorical_columns = df.select_dtypes(include='object').columns

for col in categorical_columns:
    plt.figure(figsize=(8,4))
    sns.countplot(data=df, x=col, order=df[col].value_counts().index)
    plt.xticks(rotation=90)
    plt.title(col)
    plt.show()

# Observation:
# Identify the most common categories.
```





## Q6. Correlation Heatmap)

Create a heatmap of the correlation matrix for numeric features.

- Use seaborn with proper annotations.
- Identify which features are highly correlated with the target variable (price).
- Write your observations in comment.

### PROGRAM SCREENSHOT :

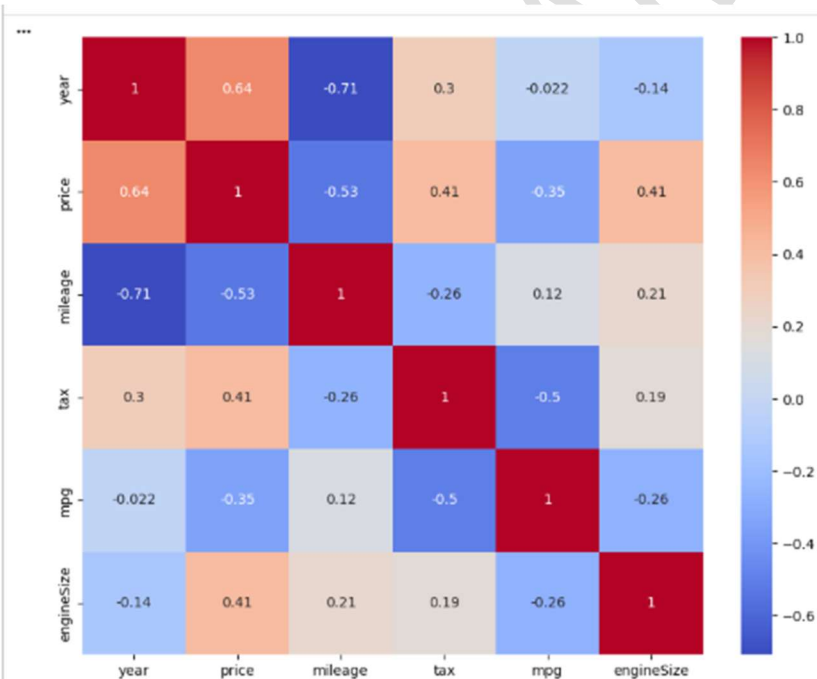
```
#Q6
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,8))

sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap='coolwarm')

plt.show()

# Observation:
# Check which features are highly correlated with Price.
```



### Q7. Feature Identification) Clearly identify and list:

- Independent Features Input Features)
- Dependent Feature Output/Target Feature)

Justify your choice with reasoning.

### PROGRAM SCREENSHOT :

```
#Q7

# Independent Features
X = df.drop('price', axis=1)

# Dependent Feature
y = df['price']

print("Independent Features")
print(X.columns)

print("\nDependent Feature")
print(y.name)

# Justification:
# Price is the target variable.

*** Independent Features
Index(['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg',
      'engineSize'],
      dtype='object')

Dependent Feature
price
```

### Q8. Encoding Categorical Variables)

Convert all categorical columns into numeric form using appropriate encoding techniques:

- Use One Hot Encoding.
- Show before and after transformation for at least 2 columns.

### PROGRAM SCREENSHOT :

```
#Q8

print("Before Encoding")

print(df[['transmission', 'fuelType']].head())

encoded_df = pd.get_dummies(df, drop_first=True)

print("\nAfter Encoding")

print(encoded_df.head())
```



```

Before Encoding
  transmission fuelType
0   Automatic   Petrol
1   Manual     Petrol
2   Manual     Petrol
3   Manual     Petrol
4   Automatic   Petrol

After Encoding
  year  price  mileage  tax  mpg  engineSize  model_ C-MAX  model_ EcoSport \
0  2017  12000   15944  150  57.7          1.0      False      False
1  2018  14000   9083  150  57.7          1.0      False      False
2  2017  13000   12456  150  57.7          1.0      False      False
3  2019  17500   10460  145  48.3          1.5      False      False
4  2019  16500   1482  145  48.7          1.0      False      False

  model_ Edge  model_ Escort  ...  model_ Tourneo Connect \
0      False      False  ...              False
1      False      False  ...              False
2      False      False  ...              False
3      False      False  ...              False
4      False      False  ...              False

  model_ Tourneo Custom  model_ Transit Tourneo  model_Focus \
0      False          False          False      False
1      False          False          False      False
2      False          False          False      False
3      False          False          False      False
4      False          False          False      False

  transmission_Manual  transmission_Semi-Auto  fuelType_Electric \
0      False          False          False      False
1      True           False          False      False
2      True           False          False      False
3      True           False          False      False
4      False          False          False      False

  fuelType_Hybrid  fuelType_Other  fuelType_Petrol
0      False      False          True
1      False      False          True
2      False      False          True
3      False      False          True
4      False      False          True

[5 rows x 35 columns]

```

## Q9. Feature Scaling)

Apply Standard Scaling (using StandardScaler from sklearn) on the numeric independent features

- . Show the first 5 rows of scaled data.

### PROGRAM SCREENSHOT :

```

#Q9

from sklearn.preprocessing import StandardScaler

numeric_columns = X.select_dtypes(include='number').columns

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X[numeric_columns])

scaled_df = pd.DataFrame(X_scaled, columns=numeric_columns)

print(scaled_df.head())

```

|   | year     | mileage   | tax      | mpg       | engineSize |
|---|----------|-----------|----------|-----------|------------|
| 0 | 0.067059 | -0.382994 | 0.591380 | -0.020597 | -0.810561  |
| 1 | 0.554393 | -0.736317 | 0.591380 | -0.020597 | -0.810561  |
| 2 | 0.067059 | -0.562616 | 0.591380 | -0.020597 | -0.810561  |
| 3 | 1.041726 | -0.665405 | 0.510777 | -1.737858 | 0.345325   |
| 4 | 1.041726 | -1.127749 | 0.510777 | -0.908836 | -0.810561  |

## Q10. Mini Project - Complete Preprocessing Pipeline)

Create a complete preprocessing pipeline for the Ford Car Dataset:

1. Load and clean the data (handle missing & duplicate values).
2. Perform EDA (histograms + count plots + heatmap).
3. Identify input and output features.
4. Encode categorical variables.

### PROGRAM SCREENSHOT :

```
#Q10

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler

# Load Dataset
data = pd.read_csv('/content/ford_car_dataset.csv')

# Remove duplicates
data.drop_duplicates(inplace=True)

# Missing values
print(data.isnull().sum())

# Histograms
data[['price', 'mileage', 'year', 'engineSize', 'mpg']].hist(figsize=(12,8))
plt.show()

# Count plots
cat_cols = data.select_dtypes(include='object').columns

for col in cat_cols:
    plt.figure(figsize=(8,4))
    sns.countplot(data=data,
                  x=col,
                  order=data[col].value_counts().index)
    plt.xticks(rotation=90)
    plt.show()

# Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(data.corr(numeric_only=True),
            annot=True,
            cmap='coolwarm')
plt.show()

# Input & Output
X = data.drop('price', axis=1)
y = data['price']

# Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(data.corr(numeric_only=True),
            annot=True,
            cmap='coolwarm')
plt.show()

# Input & Output
X = data.drop('price', axis=1)
y = data['price']

# One Hot Encoding
X = pd.get_dummies(X, drop_first=True)

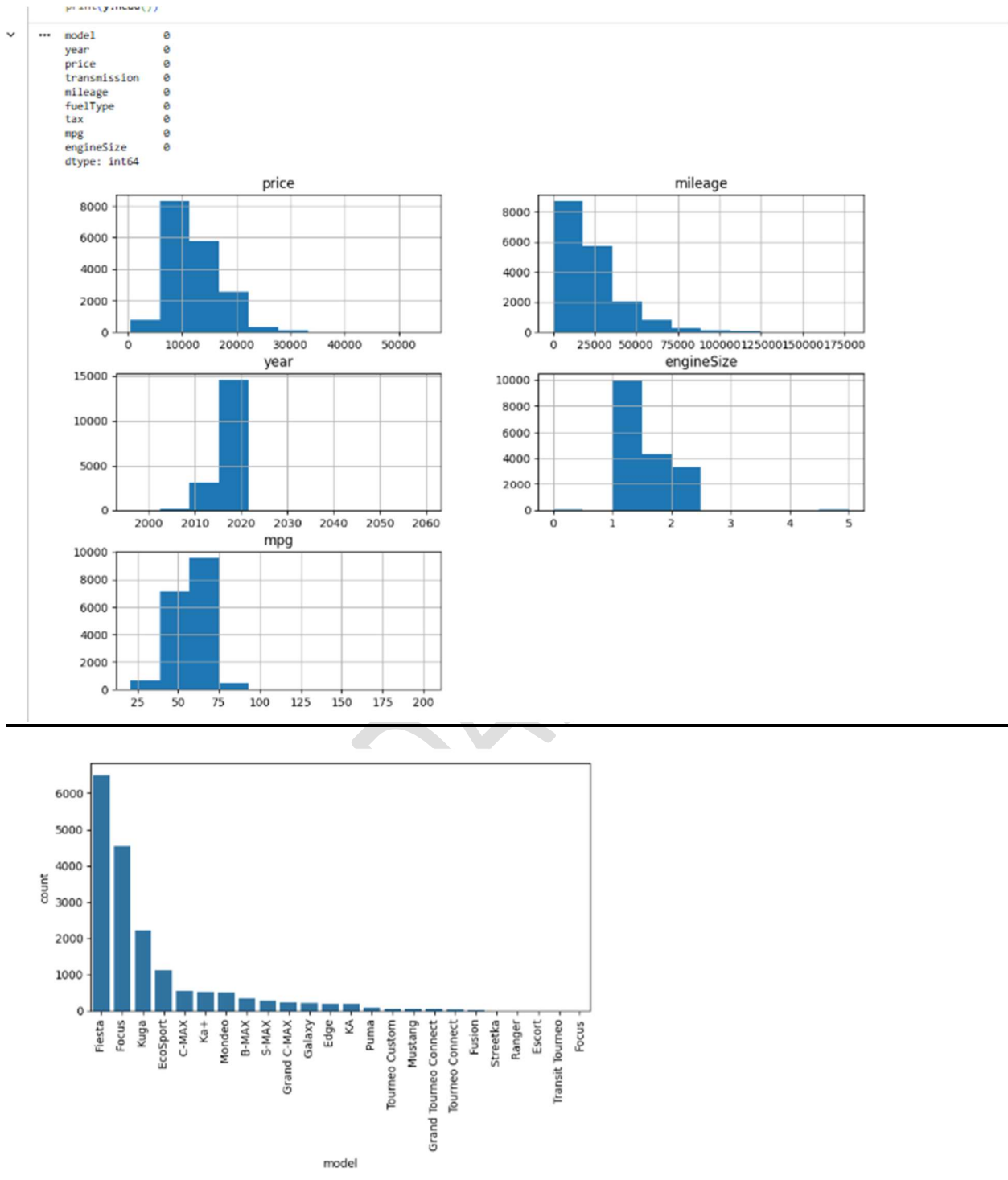
# Standard Scaling
num_cols = X.select_dtypes(include='number').columns

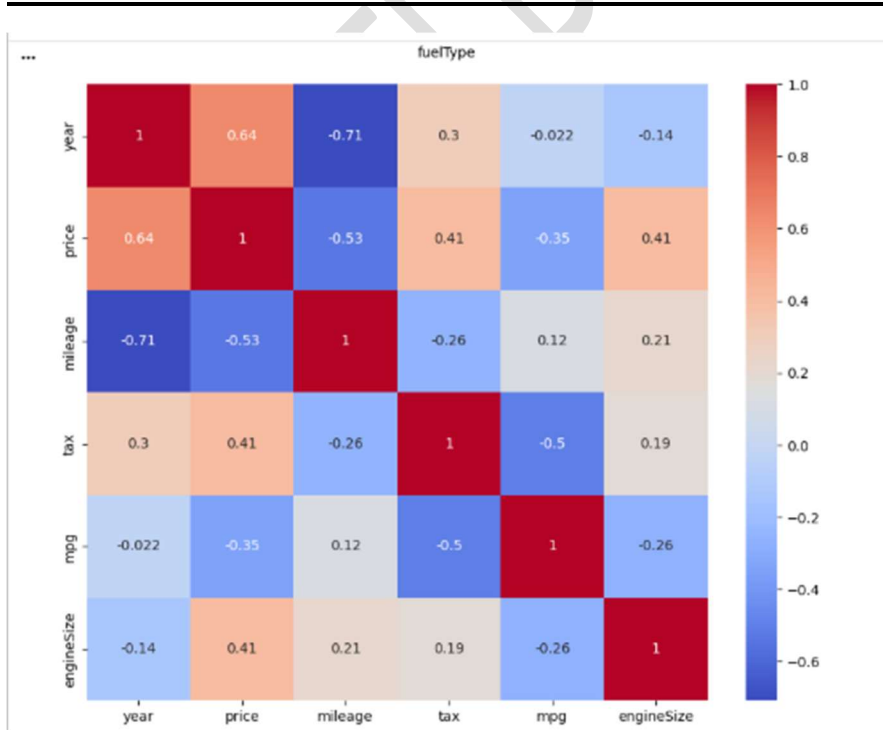
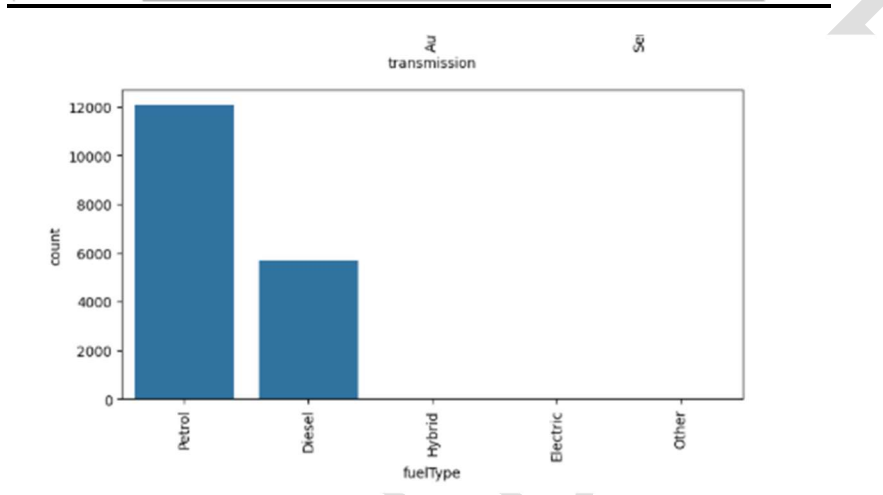
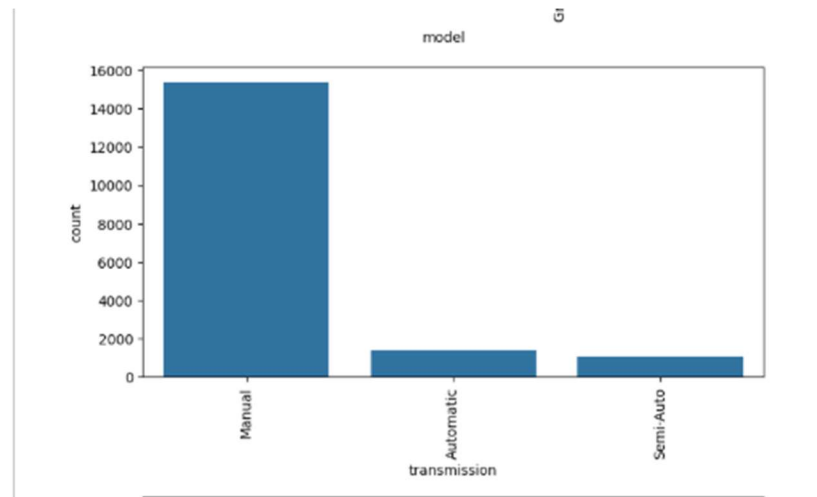
scaler = StandardScaler()

X[num_cols] = scaler.fit_transform(X[num_cols])

print("Processed Features")
print(X.head())

print("\nTarget Variable")
print(y.head())
```





|                                                                       | year     | price     | mileage  | tax       | mpg        | engineSize     |
|-----------------------------------------------------------------------|----------|-----------|----------|-----------|------------|----------------|
| Processed Features                                                    |          |           |          |           |            |                |
|                                                                       | year     | mileage   | tax      | mpg       | engineSize | model_ C-MAX \ |
| 0                                                                     | 0.067059 | -0.382994 | 0.591380 | -0.020597 | -0.810561  | False          |
| 1                                                                     | 0.554393 | -0.736317 | 0.591380 | -0.020597 | -0.810561  | False          |
| 2                                                                     | 0.067059 | -0.562616 | 0.591380 | -0.020597 | -0.810561  | False          |
| 3                                                                     | 1.041726 | -0.665405 | 0.510777 | -1.737858 | 0.345325   | False          |
| 4                                                                     | 1.041726 | -1.127749 | 0.510777 | -0.908836 | -0.810561  | False          |
| model_ EcoSport model_ Edge model_ Escort model_ Fiesta ... \         |          |           |          |           |            |                |
| 0                                                                     | False    | False     | False    | True      | ...        |                |
| 1                                                                     | False    | False     | False    | False     | False      | ...            |
| 2                                                                     | False    | False     | False    | False     | False      | ...            |
| 3                                                                     | False    | False     | False    | True      | True       | ...            |
| 4                                                                     | False    | False     | False    | True      | True       | ...            |
| model_ Tourneo Connect model_ Tourneo Custom model_ Transit Tourneo \ |          |           |          |           |            |                |
| 0                                                                     | False    | False     | False    | False     | False      |                |
| 1                                                                     | False    | False     | False    | False     | False      |                |
| 2                                                                     | False    | False     | False    | False     | False      |                |
| 3                                                                     | False    | False     | False    | False     | False      |                |
| 4                                                                     | False    | False     | False    | False     | False      |                |
| model_Focus transmission_Manual transmission_Semi-Auto \              |          |           |          |           |            |                |
| 0                                                                     | False    | False     | False    | False     | False      |                |
| 1                                                                     | False    | True      | False    | False     | False      |                |
| 2                                                                     | False    | True      | False    | False     | False      |                |
| 3                                                                     | False    | True      | False    | False     | False      |                |
| 4                                                                     | False    | False     | False    | False     | False      |                |
| fuelType_Electric fuelType_Hybrid fuelType_Other fuelType_Petrol      |          |           |          |           |            |                |
| 0                                                                     | False    | False     | False    | False     | True       |                |
| 1                                                                     | False    | False     | False    | False     | True       |                |
| 2                                                                     | False    | False     | False    | False     | True       |                |
| 3                                                                     | False    | False     | False    | False     | True       |                |
| 4                                                                     | False    | False     | False    | False     | True       |                |
| [5 rows x 34 columns]                                                 |          |           |          |           |            |                |
| Target Variable                                                       |          |           |          |           |            |                |
| 0                                                                     | 12000    |           |          |           |            |                |
| 1                                                                     | 14000    |           |          |           |            |                |
| 2                                                                     | 13000    |           |          |           |            |                |
| 3                                                                     | 17500    |           |          |           |            |                |
| 4                                                                     | 16500    |           |          |           |            |                |
| Name: price, dtype: int64                                             |          |           |          |           |            |                |